

Detailed Plan and Design for a Dual-Mode Image Upscaling and Vectorization App

Executive summary

The strongest product direction is **one app with two clearly separated modes** rather than one generic “enhance my image” button: **Upscale** for photos, AI art, UI screenshots, logos, and low-resolution graphics; and **Vectorize** for logos, icons, clipart, line art, badges, decals, and print-ready artwork. The five references you shared point to the same core lesson: users want a simple first-run experience, but the best products also expose **expert controls only when needed**. Vectorizer.AI emphasizes automatic, high-fidelity vectorization with strong export controls; Vectorizer.io emphasizes a wizard-like, highly tunable tracing workflow; Topaz emphasizes model selection/routing for different image types; Poe shows how the same capability can be wrapped in a conversational interface; and the mobile logo app shows how packaging, onboarding, and monetization shape user perception even when the underlying image tech is not the same. ¹

My highest-confidence recommendation is a **hybrid rollout**. Launch the **web app first** with cloud processing, then add **mobile clients as thin cloud front ends**, then ship a **Poe bot** as a discovery and distribution channel. For the first release, use **best-in-class external APIs or model adapters** for each mode so you can validate demand quickly; only after you see usage patterns should you invest in a proprietary vector engine or your own super-resolution stack. That is the fastest way to match the quality bar set by the five examples without taking on the full R&D cost up front. ²

The most important strategic decision is to avoid treating vectorization and upscaling as the same problem. They are not. Upscaling is an **image restoration / super-resolution** problem, where modern systems typically rely on model families such as ESRGAN, Real-ESRGAN, and SwinIR-style restoration architectures. Vectorization is a **structure extraction and SVG construction** problem, where the strongest public signals point to hybrid systems that combine segmentation, palette analysis, geometry fitting, curve cleanup, and export logic; classical bitmap tracers like Potrace still matter for line art, while newer research such as DiffVG and StarVector shows where ML-native vector pipelines are heading. ³

What the five reference apps reveal

Vectorizer.AI is the clearest signal for the quality bar on automated vectorization. Publicly, it says it converts PNG, JPG, GIF, BMP, and WebP into SVG, PDF, EPS, DXF, and PNG; supports full color and transparency; allows pre-crop; and combines deep learning with classical geometry cleanup. It explicitly says its “Deep Vector Engine” is built from in-house deep learning networks plus classical algorithms, and it mentions geometric fitting, corner cleanup, tangent matching, curve fairing, and a “Vector Graph” representation. That is unusually specific language for a commercial app and strongly suggests a **hybrid AI + geometry pipeline**, not a pure legacy tracer. ⁴

Vectorizer.io exposes a very different philosophy. Its current product surfaces model choices like **clipart, photo, sketch, and drawing**; algorithm choices like **overlap, single, contour, centerline, border**; palette editing; gradient editing; noise reduction; upscaling; crop; paint-by-numbers; and even a color-limited workflow. Its docs show that gradients remain a hard case and sometimes require manual intervention, and its API exposes many low-level controls including max colors, anti-aliasing, minimum shape area, palette import, and background handling. That is the public surface area of a product built around a **controllable tracing pipeline**, likely heavier on classical image-processing heuristics and explicit post-processing than Vectorizer.AI's more opaque auto-first positioning. ⁵

Topaz Gigapixel shows what best-in-class upscaling looks like as a product, not just as a model. Topaz markets Gigapixel as a professional image upscaler, says it offers multiple specialized models, and documents families such as **Standard 2, Low Resolution 2, Art & CGI**, and **Text & Shapes**. The published descriptions are important: Standard 2 is recommended for most photos and graphics, Low Resolution 2 is for small or heavily compressed sources, Art & CGI preserves clean edges and flat color regions, and Text & Shapes is specifically for typography, logos, UI, and geometric shapes. This is exactly the pattern your app should copy: **route users to the right model class automatically, but let advanced users override it**. ⁶

TopazLabs on Poe demonstrates that users will accept the same underlying image capability in a chat interface as long as the parameters remain simple. Poe's Topaz bot says it can upscale images by a specified factor up to 16, lets users set output width and height, and includes a toggle for AI-generated inputs. Poe's bot platform also supports image inputs, custom parameter controls, and API-backed bots, which makes Poe a viable later-stage distribution surface for your service rather than just a marketing experiment. ⁷

The **AI Logo Maker** Android app is the outlier. The Play listing describes prompting, instant logo generation, customization, and export, but the listing does **not** disclose SVG export or vectorization as a core capability. Its real value as a reference is therefore **mobile packaging**, not core imaging technology: easy onboarding, fast prompt-to-result flow, customization, and aggressive subscription monetization. That matters because recent visible Play reviews on the listing complain about trial-card friction and mismatch between ad promises and in-app reality, which is exactly the kind of trust failure your own app should avoid. ⁸

How these apps likely work under the hood

The public evidence suggests that **Vectorizer.AI** probably runs a pipeline something like this: image ingestion and normalization, semantic or contour-aware decomposition, palette / region analysis, sub-pixel boundary estimation using anti-aliased edges, shape graph construction, curve fitting, corner optimization, topology cleanup, and finally export transformation into SVG/PDF/EPS/DXF/PNG with format-specific knobs. That inference comes directly from the company's public statements about deep learning plus classical algorithms, sub-pixel precision, vector graph cleanup, grouping options, cut-outs vs stacked shapes, and export compatibility controls. ⁹

Vectorizer.io looks more like a classic but well-productized tracing engine with an AI-assisted preprocessing layer. Its home page currently advertises "Enhance image with AI (experimental)" to remove noise, simplify colors, and improve edges before vectorization. Its API and tutorials expose model selection, color count reduction, anti-aliasing decisions, minimum area thresholds, explicit palette import, and algorithm families including filled-layer and stroked-layer modes. The product's own tutorials also recommend reducing colors

and adjusting palettes for better vector output, which is a strong sign that **quantization and controllable segmentation** are central to the pipeline. ¹⁰

Topaz is public about the fact that upscaling is not one model for every case. The developer docs price the Gigapixel family at **1 credit per 24 megapixels of output**, and the API supports requested output dimensions, output formats, and model-specific extra settings. The published model descriptions show a routing strategy across photos, low-res inputs, art/CGI, text/shapes, transparency-safe workflows, and face-specific recovery. That is the clearest possible commercial pattern for your upscale mode: **Autopilot first, specialist models underneath**. ¹¹

The broader technical landscape supports this split design. On the upscale side, ESRGAN improved adversarial super-resolution with RRDB blocks, Real-ESRGAN extended that into practical real-world blind restoration trained with synthetic degradations, and SwinIR showed transformer-based restoration performance across super-resolution, denoising, and JPEG artifact reduction. On the vector side, Potrace remains a durable baseline for monochrome tracing; DiffVG introduced differentiable rasterization to bridge raster and vector optimization; and StarVector reframed image-to-SVG as SVG code generation using an image encoder plus transformer language model. In other words, the public research record supports **separate stacks for restoration and SVG generation**, not one monolithic “image AI” model. ³

Recommended product strategy

Your product should be positioned as **a production tool, not a toy**. The landing page and first-run UI should ask one simple question: **“What do you want to do with this image?”** If the answer is “make it bigger and cleaner,” route to Upscale. If the answer is “make it editable, scalable, or print/cut ready,” route to Vectorize. This mirrors the product clarity of Topaz on the upscale side and Vectorizer.AI / Vectorizer.io on the vector side. ¹²

The **Upscale** mode should be explicitly optimized for four content classes: **photo, AI-generated art, text/UI/logos**, and **transparent overlays**. That recommendation is directly grounded in Topaz’s own model segmentation across Standard 2, Low Resolution 2, Art & CGI, Text & Shapes, face recovery, and transparency-safe tools. Your app should auto-detect the likely class and choose a default model, but advanced users should be able to override the router from an “Expert” drawer. ¹³

The **Vectorize** mode should be explicitly optimized for **logos, icons, illustrations, line art, decals, stickers, embroidery/cutting prep, and print**. It should not promise good results for every photographic input. Vectorizer.AI says it can work on photos, but both its positioning and Vectorizer.io’s tutorials still indicate that graphics, logos, and drawings are the sweet spot. Your own copy should be honest: support photos as an advanced use case, but market vectorization first for structured graphics. ¹⁴

For the first public release, the most sensible commercial strategy is **API-led for quality, then selectively internalize**. Use an external upscale provider such as Topaz’s Image API for premium results and either an external vector API or a limited in-house tracing engine for the vector side. If your usage later clusters around very specific inputs such as logos with flat colors, tattoos, stickers, and icons, then build or train your own vectorizer for those narrow cases while keeping fallback adapters for difficult jobs. That is how you minimize time-to-market while keeping a road to margin improvement later. ¹⁵

System architecture and rollout

The **web app** should be the system of record. A practical stack is a React or Next.js front end, an API gateway, an auth/subscription service, a job orchestration service, GPU-capable workers for upscale jobs, CPU/GPU vector workers, object storage for originals and outputs, and a metadata database for jobs, previews, receipts, and entitlements. The upload flow should be synchronous only for validation and preview creation; production processing should be asynchronous, with progress updates via polling or websockets. This architecture matches the constraints visible in the reference APIs, which all expect authenticated uploads, rate limits, timeouts, and server-side processing. ¹⁶

Your **job model** should support two distinct pipelines. For Upscale, the job stores input metadata, inferred content type, target dimensions or factor, selected model, and output format. For Vectorize, it stores the source image, palette strategy, algorithm/mode, shape cleanup settings, gradient handling mode, grouping/export options, and whether the job is meant for visual fidelity or editability. This separation is important because the export and tuning controls are fundamentally different between the two domains, just as Vectorizer.AI's output controls differ from Topaz's resizing and model routing controls. ¹⁷

The **mobile phase** should be cloud-first. Topaz's own ecosystem leans heavily on cloud rendering for large or generative jobs, and its pricing pages now package image cloud rendering as part of larger subscriptions. That is a strong market signal that heavy image enhancement should not be treated as purely on-device by default. Your mobile app should therefore focus on import/capture, queueing, presets, saved jobs, share/export, and mobile-friendly review rather than trying to replicate the full compute stack locally. ¹⁸

For the **Poe bot**, I would ship a **server bot first** and only consider an API Bot later. Poe server bots let you accept file uploads through attachments and return inline images or file attachments in the response, which is ideal for your transform workflow. API Bots are also viable if your backend already exposes a Chat Completions or Responses API-compatible endpoint, and they support input modalities plus custom parameter controls, but the image-transform use case maps especially cleanly to the server-bot attachment flow. ¹⁹

UX, monetization, and trust

The best UX pattern across your references is **preview before payment**. Vectorizer.AI lets users upload, process, and inspect results before they subscribe; its API even supports discounted preview workflows. Vectorizer.io also offers evaluation credits and a web flow that clearly distinguishes free evaluation from paid downloading and API use. Your app should copy this pattern: free upload, free preview, free side-by-side compare, then charge only for premium download/export or higher-volume usage. ²⁰

Monetization should differ by channel. On the web, use **free previews + paid downloads/subscription**. For API customers, use **credits or pay-per-job**, which mirrors both Vectorizer.AI and Topaz's pricing logic. On Poe, billing will need to fit Poe's point economy, where usage is charged according to configured pricing and points availability. On mobile, be conservative with subscriptions and very explicit about trials, because the reference logo app's recent listing reviews show how quickly unclear trial mechanics can create user distrust. ²¹

Trust and privacy should be treated as product features. One reason to keep your own backend in front of any third-party processor is that vendor terms and data practices vary. Vectorizer.AI's current Terms of Use say user submissions and results expire after two weeks, but also state that content may be retained and used for model training by the company by default unless a particular site or service says otherwise. The AI Logo Maker app's privacy policy says it may process uploaded images, prompts, and files and may use third-party providers for hosting, analytics, and image processing. If your app is going to handle user brand assets, client artwork, or confidential files, your own product should promise clearer retention defaults and stronger deletion controls than many off-the-shelf AI tools do. ²²

Operationally, you should build around **quality routing and failure recovery**. Vectorization jobs should fail gracefully into "This image may be better upscaled first" or "This image may need fewer colors / crop / transparency flattening." Upscale jobs should fall back from generative/high-cost models to fidelity-first models when the input is already clean. The references show that quality depends as much on **correct mode selection** as on raw model power. ²³

Build versus buy decision

If your goal is to launch quickly and match the quality users already see in these products, the best near-term choice is **buy the hard parts, own the experience**. Concretely, that means a branded web app and backend that handle uploads, previews, billing, storage, analytics, presets, batch jobs, and export management, while the image transformations themselves come from premium providers or tightly wrapped model services. The reference vendors all provide public API or bot-compatible surfaces, which is exactly what enables this strategy. ²⁴

If your goal is long-term differentiation and better margins, the order of internalization should be **vectorization second, not first**. Upscaling can be covered reasonably well by existing model families and routing strategies, and public research/open implementations are mature enough to build a competent stack around. But full-color, edit-friendly, commercially robust vectorization at the quality level implied by Vectorizer.AI appears significantly harder because it requires not just segmentation, but also topology, geometry, cleanup, and export semantics. In other words: building a "good enough" upscaler is more straightforward than building a "good enough" full-color vectorizer that users trust for logos and production graphics. ²⁵

My recommendation is therefore a **three-step roadmap**. First, ship an API-backed web product with superb UX and transparent pricing. Second, add mobile clients that reuse the same backend and presets. Third, add a Poe server bot for discovery and conversational workflows. Only then, if usage data justifies it, start replacing the most expensive third-party transformations with in-house models or narrower specialized pipelines for your top input categories. ²⁶

Open questions and limitations

Some important details remain inherently uncertain because the strongest commercial products here are proprietary. I can make high-confidence inferences about their architecture from their public docs and UI surfaces, but I cannot verify their training data composition, exact model architectures, or the precise balance between learned and heuristic components beyond what they publicly disclose. That applies especially to Vectorizer.AI and Topaz's internal pipelines. ²⁷

The mobile reference is also only a **partial analog** for your product. Its Play listing demonstrates mobile AI onboarding and monetization patterns, but it does not document SVG export or raster-to-vector conversion as a core capability. I would treat it as a reference for mobile UX and packaging, not for the actual vectorization engine you want to build. ²⁸

This response contains the **Markdown master report in-chat**. **DOCX and PDF download files are not attached in this response.**

¹ ⁴ ¹² <https://vectorizer.ai/>

<https://vectorizer.ai/>

² ²⁰ ²¹ <https://vectorizer.ai/pricing>

<https://vectorizer.ai/pricing>

³ ²⁵ <https://arxiv.org/abs/1809.00219>

<https://arxiv.org/abs/1809.00219>

⁵ <https://www.vectorizer.io/api/>

<https://www.vectorizer.io/api/>

⁶ <https://www.topazlabs.com/topaz-gigapixel>

<https://www.topazlabs.com/topaz-gigapixel>

⁷ <https://poe.com/TopazLabs>

<https://poe.com/TopazLabs>

⁸ ²⁸ <https://play.google.com/store/apps/details?id=com.logomaker.logo.creator>

<https://play.google.com/store/apps/details?id=com.logomaker.logo.creator>

⁹ ¹⁴ ²⁷ <https://api.vectorizer.ai/>

<https://api.vectorizer.ai/>

¹⁰ <https://www.vectorizer.io/>

<https://www.vectorizer.io/>

¹¹ <https://developer.topazlabs.com/image-models/gigapixel>

<https://developer.topazlabs.com/image-models/gigapixel>

¹³ ²³ <https://developer.topazlabs.com/image-models/gigapixel/standard-2>

<https://developer.topazlabs.com/image-models/gigapixel/standard-2>

¹⁵ <https://developer.topazlabs.com/getting-started/quickstart>

<https://developer.topazlabs.com/getting-started/quickstart>

¹⁶ ²⁴ <https://vectorizer.ai/api>

<https://vectorizer.ai/api>

¹⁷ <https://vectorizer.ai/api/outputOptions>

<https://vectorizer.ai/api/outputOptions>

¹⁸ <https://docs.topazlabs.com/topaz-gigapixel/cloud-rendering>

<https://docs.topazlabs.com/topaz-gigapixel/cloud-rendering>

19 26 <https://creator.poe.com/docs/server-bots/enabling-file-upload-for-your-bot>
<https://creator.poe.com/docs/server-bots/enabling-file-upload-for-your-bot>

22 <https://vectorizer.ai/policies/terms>
<https://vectorizer.ai/policies/terms>